REFERENCE · CHAPTERS 4 & 6

Workload Identity (SPIFFE), mTLS, DPoP & OAuth 2.0 OBO Token Exchange

How every hop between the User, Agent, Egress Gateway, and MCP Tool Server proves both workload identity and delegated human authority.

1. The Four Building Blocks of Hop-by-Hop Agent Identity

When an AI agent calls a database tool server on behalf of a human user, two distinct identities must be proven on every network hop: (1) **Workload Identity** (which agent process is making the call) and (2) **Delegated User Identity** (which human user authorized the task, and for which target audience).

LAYER	MECHANISM	WHAT IT PROVES	THREAT CLOSED
Workload Identity	**SPIFFE ID + X.509 SVID** (`principal://agents.global…/reasoningEngines/AGENT_ID`)	Issued by the Agent Runtime control plane at startup; zero static JSON keys in environment variables or container layers.	`T3` (Credential theft)
Transport Layer	**Mutual TLS (mTLS)**	Both client (agent) and server (Egress Gateway / MCP server) verify each other's X.509 certificates during the TLS handshake.	`T5` / `ASI07` (Spoofed caller)
Proof-of-Possession	**DPoP (`RFC 9449`)**	Binds the OAuth token to the agent's private key via a per-request signed `DPoP` HTTP header covering `htm` , `htu` , and `ath` .	`T5` (Stolen bearer token replay)
Delegated Token Hop	**OAuth 2.0 OBO Token Exchange (`RFC 8693`)** or **KMS-Signed Context JWT**	Swaps inbound token `Token_in` (`aud=web-app`) for a 5-minute, audience-locked token `Token_obo`	`T4` / `T5` (Token passthrough & BOLA)

LAYER	MECHANISM	WHAT IT PROVES	THREAT CLOSED
		(`aud=orders-mcp` , `scp=orders:read` , `act={sub: agent-a}`).	

obo_token_exchange_and_verify.py — RFC 8693 Token Exchange & Downstream MCP Verification

```python
import requests
from google.auth.transport import requests as google_requests
from google.oauth2 import id_token

STS_URL = "https://sts.googleapis.com/v1/token"
EXPECTED_MCP_AUD = "https://orders-mcp-xyz-uc.a.run.app"
ALLOWED_AGENT_ACTORS = {"spiffe://agents.global.org-123.system.id.goog

def exchange_user_token_for_mcp_target(subject_jwt: str, target_aud: s
    # Exchanges inbound Token_in (aud=web-app) for downscoped 5-min To
    resp = requests.post(
        STS_URL,
        json={
            "grantType": "urn:ietf:params:oauth:grant-type:token-excha
            "resource": target_aud,
            "audience": target_aud,
            "scope": scope,
            "requestedTokenType": "urn:ietf:params:oauth:token-type:ac
            "subjectToken": subject_jwt,
            "subjectTokenType": "urn:ietf:params:oauth:token-type:jwt"
        },
        timeout=(2, 5),
    )
    resp.raise_for_status()
    return resp.json()["access_token"]

def verify_obo_token_on_mcp_server(auth_header: str) -> dict:
    # Verifies Token_obo signature, expiry, exact audience (rejects To
```

```
    raw_jwt = auth_header.removeprefix("Bearer ").strip()
    claims = id_token.verify_oauth2_token(raw_jwt, google_requests.Re
    actor = claims.get("act", {}).get("sub")
    if actor not in ALLOWED_AGENT_ACTORS:
        raise PermissionError(f"Unauthorized acting agent: {actor}")
    return {"user_sub": claims["sub"], "user_email": claims["email"],
```